Final Project-ADS 500B

Audrey Dilgarde

2025-12-08

Contents

0.1 Importing The Dataset . . . 1
0.2 Dataset Overview . . . 2
0.3 Data Cleaning . . . 3
0.4 Feature Construction . . . 3
0.5 Aggregation . . . 4
0.6 Reducing Redundant Variables . . . 6
0.7 Discretization . . . 7
0.8 Identifying Categorical, Ordinal, and Numerical Variables . . . 7
0.9 Summary Statistics For Desirability Variables . . . 8
0.10 Desirability Distributions . . . 9
0.11 Correlations & Identifying Independent and Dependent Variables . . . 15
0.12 Unsupervised Learning: Does Desirability Garner A Premium? . . . 20
0.13 References . . . 23
0.14 Artifical Intelligence Disclosure . . . 24

0.1 Importing The Dataset

I imported the 'house_sales.csv' file into R using 'read.csv()' and stored it into the data frame 'house_sales'.

```
house_sales <- read.csv("house_sales.csv")
head(house_sales)
```

```
##           id            date   price bedrooms bathrooms sqft_living sqft_lot
## 1 7129300520 20141013T000000  221900        3      1.00        1180     5650
## 2 6414100192 20141209T000000  538000        3      2.25        2570     7242
## 3 5631500400 20150225T000000  180000        2      1.00         770    10000
## 4 2487200875 20141209T000000  604000        4      3.00        1960     5000
## 5 1954400510 20150218T000000  510000        3      2.00        1680     8080
## 6 7237550310 20140512T000000 1225000        4      4.50        5420   101930
##   floors waterfront view condition grade sqft_above sqft_basement yr_built
## 1      1          0    0         3     7       1180             0     1955
```

```
## 2      2          0    0         3     7       2170           400     1951
## 3      1          0    0         3     6        770             0     1933
## 4      1          0    0         5     7       1050           910     1965
## 5      1          0    0         3     8       1680             0     1987
## 6      1          0    0         3    11       3890          1530     2001
##   yr_renovated zipcode     lat     long sqft_living15 sqft_lot15
## 1            0   98178 47.5112 -122.257          1340       5650
## 2         1991   98125 47.7210 -122.319          1690       7639
## 3            0   98028 47.7379 -122.233          2720       8062
## 4            0   98136 47.5208 -122.393          1360       5000
## 5            0   98074 47.6168 -122.045          1800       7503
## 6            0   98053 47.6561 -122.005          4760     101930
```

0.2 Dataset Overview

In this step, I began by examining the structure of the house_sales dataset to understand the types of variables and overall dimensions. I then ran summary commands to identify which columns contained missing values and how many missing entries each column had. This allowed me to see that four columns (bedrooms, bathrooms, sqft_living, and sqft_lot) contain missing values with a total of 4,356 missing entries, all of which are numerical variables. This infomation will guide my data-cleaning decisions in the next section.

```r
str(house_sales)
```

```
## 'data.frame':    21613 obs. of  21 variables:
##  $ id           : num  7129300520 6414100192 5631500400 2487200875 1954400510 ...
##  $ date         : chr  "20141013T000000" "20141209T000000" "20150225T000000" "20141209T000000" ...
##  $ price        : num  221900 538000 180000 604000 510000 ...
##  $ bedrooms     : num  3 3 2 4 3 4 3 3 3 3 ...
##  $ bathrooms    : num  1 2.25 1 3 2 4.5 2.25 1.5 1 2.5 ...
##  $ sqft_living  : num  1180 2570 770 1960 1680 ...
##  $ sqft_lot     : num  5650 7242 10000 5000 8080 ...
##  $ floors       : num  1 2 1 1 1 1 2 1 1 2 ...
##  $ waterfront   : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ view         : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ condition    : int  3 3 3 5 3 3 3 3 3 3 ...
##  $ grade        : int  7 7 6 7 8 11 7 7 7 7 ...
##  $ sqft_above   : int  1180 2170 770 1050 1680 3890 1715 1060 1050 1890 ...
##  $ sqft_basement: int  0 400 0 910 0 1530 0 0 730 0 ...
##  $ yr_built     : int  1955 1951 1933 1965 1987 2001 1995 1963 1960 2003 ...
##  $ yr_renovated : int  0 1991 0 0 0 0 0 0 0 0 ...
##  $ zipcode      : int  98178 98125 98028 98136 98074 98053 98003 98198 98146 98038 ...
##  $ lat          : num  47.5 47.7 47.7 47.5 47.6 ...
##  $ long         : num  -122 -122 -122 -122 -122 ...
##  $ sqft_living15: int  1340 1690 2720 1360 1800 4760 2238 1650 1780 2390 ...
##  $ sqft_lot15   : int  5650 7639 8062 5000 7503 101930 6819 9711 8113 7570 ...
```

```r
missing_counts <- colSums(is.na(house_sales))
missing_counts
```

```
##            id          date         price      bedrooms     bathrooms
##             0             0             0          1134          1068
##   sqft_living      sqft_lot        floors    waterfront          view
```

```
##          1110          1044             0             0             0
##     condition         grade    sqft_above sqft_basement      yr_built
##             0             0             0             0             0
##  yr_renovated       zipcode           lat          long sqft_living15
##             0             0             0             0             0
##    sqft_lot15
##             0
```

```
number_columns_with_counts <- sum(missing_counts > 0)
number_columns_with_counts
```

```
## [1] 4
```

```
total_missing_count <- sum(missing_counts)
total_missing_count
```

```
## [1] 4356
```

0.3 Data Cleaning

Because all missing values occur in numeric variables, I decided to replace them with the median of their respective columns. This is preferable to replacing them with 0, which would incorrectly imply that some homes have 0 bathrooms, bedrooms, or square footage. The median is a robust measure of central tendency that is not heavily influenced by outliers, so this replacement preserves the typical value of each variable while only slightly altering the shape of the distribution.

```
columns_with_missing_values <- c("bedrooms", "bathrooms",
                                 "sqft_living", "sqft_lot")

for (col in columns_with_missing_values) {

  median_house_sales <- median(house_sales[[col]], na.rm = TRUE)
  house_sales[[col]][is.na(house_sales[[col]])] <- median_house_sales
}

colSums(is.na(house_sales[ , columns_with_missing_values]))
```

```
##    bedrooms   bathrooms sqft_living    sqft_lot
##           0           0           0           0
```

0.4 Feature Construction

For feature engineering, I created new variables to better capture the desirability and condition of the homes. First, I constructed bed_bath_total (bedrooms + bathrooms) and bath_per_bed (bathrooms divided by bedrooms) to summarize the overall comfort and potential crowdedness of each homes. Second, I defined premium_view and premium_view_score to indicate whether a home has any attractive view (waterfront or non-zero view rating) and to assign a simple view "score." Finally, I derived year_sold, age, renovated, and years_since_reno to measure how old each property was at the time of sale and how recently it had been updated. These engineered features provide more interpretable measures of quality and desirrability that I can use in my clustering and visualization steps.

```r
house_sales$bed_bath_total <- house_sales$bedrooms + house_sales$bathrooms

house_sales$bath_per_bed <- ifelse(house_sales$bedrooms >
                                     0, house_sales$bathrooms /
                                     house_sales$bedrooms, NA)

house_sales$premium_view <- ifelse(house_sales$waterfront ==
                                     1 | house_sales$view > 0, 1, 0)

house_sales$premium_view_score <- 2 * house_sales$waterfront + house_sales$view

house_sales$year_sold <- as.numeric(substr(house_sales$date, 1, 4))

house_sales$age <- house_sales$year_sold - house_sales$yr_built

house_sales$renovated <- ifelse(house_sales$yr_renovated > 0, 1, 0)

house_sales$years_since_reno <- ifelse(house_sales$yr_renovated >
                                         0, house_sales$year_sold -
                                         house_sales$yr_renovated, NA)

head(house_sales[, c("bed_bath_total", "bath_per_bed", "premium_view",
                     "premium_view_score", "year_sold", "age", "renovated",
                     "years_since_reno")])
```

```
##   bed_bath_total bath_per_bed premium_view premium_view_score year_sold age
## 1           4.00    0.3333333            0                  0      2014  59
## 2           5.25    0.7500000            0                  0      2014  63
## 3           3.00    0.5000000            0                  0      2015  82
## 4           7.00    0.7500000            0                  0      2014  49
## 5           5.00    0.6666667            0                  0      2015  28
## 6           8.50    1.1250000            0                  0      2014  13
##   renovated years_since_reno
## 1         0               NA
## 2         1               23
## 3         0               NA
## 4         0               NA
## 5         0               NA
## 6         0               NA
```

0.5 Aggregation

To study the neighborhood-level patterns in home desirability, I aggregated the house-level data to the zipcode level. For each zipcode I computed the mean and median sale price, the average combined number of bedrooms and bathrooms, the average premium view score, the share of homes with any premium view, and the total number of recorded sales. This procuded a zipcode_summary table ordered from the most to least expensive zipcodes by mean price.

This aggregation lets me compare "wealthy" neighborhoods on several interpretable dimensions: 1. how expensive homes are on average, 2. how large and comfortable they tend to be (via mean_bed_bath_total), 3. and how common premium views are (premium_view_share and mean_premium_score). For exmaple, zipcodes 98039, 98004, and 98040 sit at the top of the distribution by mean price; these neighborhoods tend

to have larger homes suggesting clusters of high-end housing markets that I can sutdy in further in this investigation.

```
zipcode_summary <- house_sales %>% group_by(zipcode) %>%
  summarise(mean_price = mean(price), median_price = median(price),
            mean_bed_bath_total = mean(bed_bath_total), mean_premium_score = mean(premium_view_score), 
            n_sales = n()) %>% arrange(desc(mean_price))

print(zipcode_summary, n = Inf)
```

```
## # A tibble: 70 x 7
##    zipcode mean_price median_price mean_bed_bath_total mean_premium_score
##      <int>      <dbl>        <dbl>               <dbl>              <dbl>
##  1   98039   2160607.     1892500                 7.2             0.48
##  2   98004   1355927.     1150000                 6.35            0.312
##  3   98040   1194230.      993750                 6.67            0.869
##  4   98112   1095499.      915000                 5.84            0.149
##  5   98102    901258.      720000                 5.46            0.171
##  6   98109    879624.      736000                 5.36            0.477
##  7   98105    862825.      675000                 5.71            0.332
##  8   98006    859685.      760184.                6.34            0.651
##  9   98119    849448.      744975                 5.47            0.549
## 10   98005    810165.      765475                 6.24            0.0952
## 11   98033    803720.      678350                 5.87            0.317
## 12   98199    791821.      689800                 5.40            0.558
## 13   98075    790577.      739999                 6.55            0.373
## 14   98074    685606.      642000                 6.07            0.209
## 15   98077    682775.      652475                 6.03            0.0101
## 16   98053    678163.      635000                 5.63            0.0667
## 17   98177    676185.      554000                 5.50            0.824
## 18   98008    645507.      545000                 5.80            0.541
## 19   98052    645231.      615000                 5.93            0.0749
## 20   98122    634360.      572000                 5.24            0.210
## 21   98115    619901.      567000                 5.08            0.204
## 22   98116    618634.      562750                 5.05            0.633
## 23   98007    617105.      555000                 5.95            0.0284
## 24   98027    616991.      570500                 6.00            0.170
## 25   98029    612654.      575000                 6.00            0.0810
## 26   98144    594548.      450000                 5.26            0.443
## 27   98103    584919.      550000                 5.01            0.145
## 28   98024    580527.      460000                 5.52            0.235
## 29   98107    579053.      529950                 5.05            0.211
## 30   98117    576795.      544000                 4.84            0.183
## 31   98072    569958.      515000                 5.67            0.0586
## 32   98136    551689.      489950                 4.80            0.669
## 33   98065    527961.      500000                 6.03            0.290
## 34   98034    521653.      445950                 5.60            0.224
## 35   98059    493553.      435000                 6.02            0.0427
## 36   98011    490351.      470000                 5.79            0.0615
## 37   98070    487480.      463750                 4.69            1.49
## 38   98125    469456.      425000                 5.02            0.290
## 39   98166    464232.      390000                 5.26            0.783
## 40   98028    462480.      445000                 5.59            0.170
## 41   98014    455617.      415000                 5.04            0.185
```

```
## 42   98045    439471.      399500                 5.33            0.267
## 43   98019    424789.      401250                 5.61            0.0211
## 44   98126    424706.      395750                 4.55            0.379
## 45   98155    423726.      375000                 5.15            0.197
## 46   98010    423666.      360000.                5.33            0.100
## 47   98056    420891.      380000                 5.47            0.190
## 48   98118    417637.      367500                 4.91            0.270
## 49   98133    386997.      375000                 4.90            0.0202
## 50   98038    366868.      342000                 5.77            0.0271
## 51   98146    359483.      305000                 4.77            0.458
## 52   98108    355679.      342500                 5.10            0.0968
## 53   98058    353609.      335000                 5.59            0.0220
## 54   98092    334921.      309780                 5.71            0.137
## 55   98106    319581.      315000                 4.91            0.0746
## 56   98022    315709.      279500                 5.06            0.628
## 57   98042    311632.      292000                 5.51            0.0328
## 58   98178    310613.      278277                 5.06            0.603
## 59   98055    304262.      294950                 5.17            0.104
## 60   98198    302879.      265000                 4.94            0.664
## 61   98031    300540.      288200                 5.61            0.00730
## 62   98030    296188.      282255                 5.66            0.0234
## 63   98003    294111.      267475                 5.40            0.214
## 64   98188    289078.      264000                 5.31            0.147
## 65   98023    286733.      268450                 5.48            0.138
## 66   98148    284909.      278000                 4.95            0
## 67   98001    280805.      260000                 5.39            0.102
## 68   98032    251296.      249000                 5.10            0.0800
## 69   98168    240328.      235000                 4.46            0.0149
## 70   98002    234284.      235000                 5.15            0.0101
## # i 2 more variables: premium_view_share <dbl>, n_sales <int>
```

0.6 Reducing Redundant Variables

To reduce redundant data, I removed variables that were either pure identifiers or made obsolete by my engineered features. Specifically, I dropped id, date (replaced by year_sold), and yr_renovated (replaced by the renovated and years_since_reno). I also reviewed waterfront and view as these were combined into the more interpretable premium_view and premium_view_Score. All other variables were retained for subsequent visualization, correlation, and regression analyses.

```r
house_sales <- house_sales %>% select(-id,-date,-yr_renovated,-waterfront,-view)
names(house_sales)
```

```
##  [1] "price"              "bedrooms"           "bathrooms"
##  [4] "sqft_living"        "sqft_lot"           "floors"
##  [7] "condition"          "grade"              "sqft_above"
## [10] "sqft_basement"      "yr_built"           "zipcode"
## [13] "lat"                "long"               "sqft_living15"
## [16] "sqft_lot15"         "bed_bath_total"     "bath_per_bed"
## [19] "premium_view"       "premium_view_score" "year_sold"
## [22] "age"                "renovated"          "years_since_reno"
```

0.7 Discretization

To perform need-based discretization, I converted the continuous zipcode-level mean price into a binary category. Using the 90th percentile of mean_price as a threshold, I created a wealthy_neighborhoods variable that labels zipcodes in the top 10% of mean prices as "wealthy (top 10%)" and the remaining zipcodes as "non-wealthy (bottom 90%)." This tiering allows me to compare characteristics of wealthy neighborhoods to all others in the subsequent analysis.

```
zipcode_summary <- zipcode_summary %>%
  mutate(wealthy_neighborhoods =
           ifelse(mean_price >= quantile(mean_price, 0.9),
                  "wealthy (top 10%)", "non-wealthy (bottom 90%)"))
table(zipcode_summary$wealthy_neighborhoods)
```

```
##
## non-wealthy (bottom 90%)        wealthy (top 10%)
##                       63                        7
```

0.8 Identifying Categorical, Ordinal, and Numerical Variables

Variables are classified by how they are measured and their intended use.

Numerical variables represent quantities where differences and ratios are meaninful. In my data, price, sqft_living, sqft_lot, sqft_above, sqft_basement, sqft_living15, sqft_lot15, bed_bath_total, bath_per_bed, age, year_sold, and premium_view_score are all numerical because they measure continuous or count-like quantities (dollars, square footage, years, number of rooms). The engineered variable years_since_reno is also numerical, because it counts how many years have passed since renovation for homes that were renovated.

Ordidnal variables are coded as integers, but the numbers represent ordered levels rather than precise distances. Both grade and condition fall into this category: a higher value indicates a better quality home but the difference betwwen grades 7 and 8 is not guaranteed to be the same as between grades 8 and 9. I therefore treat these as ordered ratings rather than true continuous measurements.

Categorical variables represent labels or group membership where the numeric codes themselves have no quantitative meaning. zipcode is categorical because each categorical because each code identifies a geographic area but "98178" ia not "greather than" "98028" in a numerical sense. The indicators premium_view and renovated are binary categorical variables (0/1) showing absence or presence of a property feature. My discretized wealthy_neighborhoods variable is also categorical, assigning each zipcode to "wealthy (top 10%)" or "non-wealthy (bottom 90%)" for later group comparisons.

For the engineered years_since_reno feature, I only compute values for homes that were actually renovated. Properties with yr_renovated = 0 are coded as NA in years_since_reno, indicating that "years since renovation" is not applicable rather than truly missing data. In contrast, geniune missing values in the original numeric variables (bedrooms, bathrooms, sqft_living, sqft_lot) were imputed using the median, as described in the data cleaning section.

```
str(house_sales)
```

```
## 'data.frame':    21613 obs. of  24 variables:
##  $ price             : num  221900 538000 180000 604000 510000 ...
##  $ bedrooms          : num  3 3 2 4 3 4 3 3 3 3 ...
##  $ bathrooms         : num  1 2.25 1 3 2 4.5 2.25 1.5 1 2.5 ...
##  $ sqft_living       : num  1180 2570 770 1960 1680 ...
```

```
##  $ sqft_lot          : num  5650 7242 10000 5000 8080 ...
##  $ floors            : num  1 2 1 1 1 1 2 1 1 2 ...
##  $ condition         : int  3 3 3 5 3 3 3 3 3 3 ...
##  $ grade             : int  7 7 6 7 8 11 7 7 7 7 ...
##  $ sqft_above        : int  1180 2170 770 1050 1680 3890 1715 1060 1050 1890 ...
##  $ sqft_basement     : int  0 400 0 910 0 1530 0 0 730 0 ...
##  $ yr_built          : int  1955 1951 1933 1965 1987 2001 1995 1963 1960 2003 ...
##  $ zipcode           : int  98178 98125 98028 98136 98074 98053 98003 98198 98146 98038 ...
##  $ lat               : num  47.5 47.7 47.7 47.5 47.6 ...
##  $ long              : num  -122 -122 -122 -122 -122 ...
##  $ sqft_living15     : int  1340 1690 2720 1360 1800 4760 2238 1650 1780 2390 ...
##  $ sqft_lot15        : int  5650 7639 8062 5000 7503 101930 6819 9711 8113 7570 ...
##  $ bed_bath_total    : num  4 5.25 3 7 5 8.5 5.25 4.5 4 5.5 ...
##  $ bath_per_bed      : num  0.333 0.75 0.5 0.75 0.667 ...
##  $ premium_view      : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ premium_view_score: num  0 0 0 0 0 0 0 0 0 0 ...
##  $ year_sold         : num  2014 2014 2015 2014 2015 ...
##  $ age               : num  59 63 82 49 28 13 19 52 55 12 ...
##  $ renovated         : num  0 1 0 0 0 0 0 0 0 0 ...
##  $ years_since_reno  : num  NA 23 NA NA NA NA NA NA NA NA ...
```

0.9 Summary Statistics For Desirability Variables

I first computed summary statistics for variables tied to home desirability: price, overall size (bed_bath_total and bath_per_bed), age, view quality (premium_view_score), and years_since_reno for renovated homes. This provides measures of central tendency (mean/median) and spread (quartiles, min/max) before looking at relationships between variables.

```r
desirability_stats <-  house_sales %>% select(price, bed_bath_total,
bath_per_bed, age, premium_view_score, years_since_reno)

summary(desirability_stats)
```

```
##      price         bed_bath_total    bath_per_bed         age
##  Min.   :  75000   Min.   : 0.000   Min.   :0.0000   Min.   : -1.00
##  1st Qu.: 321950   1st Qu.: 4.500   1st Qu.:0.5000   1st Qu.: 18.00
##  Median : 450000   Median : 5.500   Median :0.6250   Median : 40.00
##  Mean   : 540088   Mean   : 5.474   Mean   :0.6476   Mean   : 43.32
##  3rd Qu.: 645000   3rd Qu.: 6.500   3rd Qu.:0.7500   3rd Qu.: 63.00
##  Max.   :7700000   Max.   :34.750   Max.   :2.6667   Max.   :115.00
##                                     NA's   :11
##  premium_view_score years_since_reno
##  Min.   :0.0000     Min.   :-1.00
##  1st Qu.:0.0000     1st Qu.: 7.00
##  Median :0.0000     Median :15.00
##  Mean   :0.2494     Mean   :18.44
##  3rd Qu.:0.0000     3rd Qu.:27.00
##  Max.   :6.0000     Max.   :80.00
##                     NA's   :20699
```

0.10 Desirability Distributions

Across all six histograms, the distributions are strongly right-skewed: most homes cluster in a fairly typical range, with a small number of extremely "fancy" cases that stretch my along the right tail. This pattern is most dramatic for sale price, where the bulk of houses under 700K-800K USD, but a small set of mult-million-dollar properties pushes the maximum towards 8 million USD. This aligns with the summary statistics, where the mean price is higher than the median.

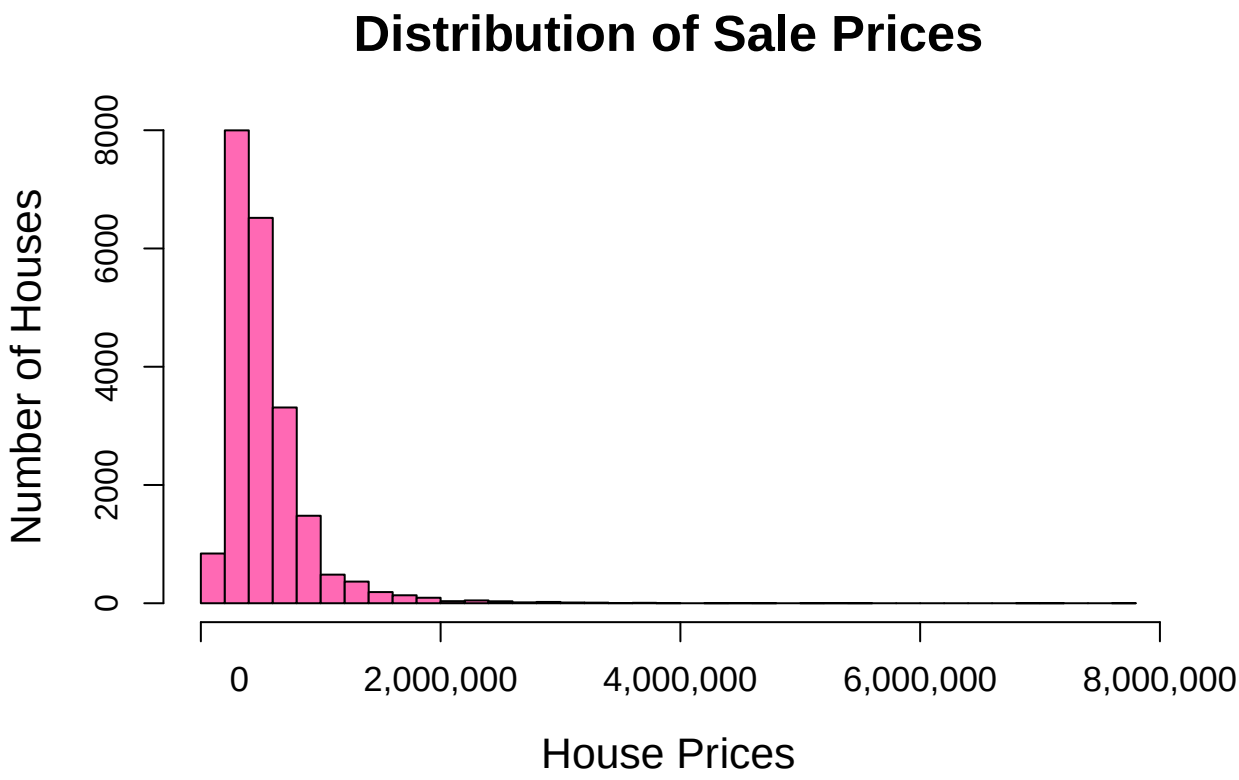
The bed_bath_total histogram shows that most homes offer between 4 and 7 combined bedrooms and bathrooms, with a very thin tail of very large houses (over 15 total rooms) that likely correspond to luxury properties. The bath_per_bed distribution is tightly 0.5 and 1.0, indicating that a "normal" configuratio for a home within the dataset is about one bathroom for every 1-2 bedrooms, with only a few outlier homes that are unusually generous in bathroom cout.

For age, after setting a handful of negative values to 0 (due to data issues where year_sold and yr_built were inconsistent), most homes are between 0 to 60 years old, with a decreasing tail that stops at 120 years. This suggests a housing stock that is mostly mid-century (1935-1975) or newer, with a smaller share of very old homes that may be historically or architecturally distinctive.

The premium_view_score histogram highlights how rare premium views actually are. The vast majority of homes have a score of 0, with progressively fewer properties at scores of 1 to 6. In other words, waterfront and high-end view ratings are concentrated in a relatively small share of the market, which fits the idea that views are a scarcity feature that can command higher prices.

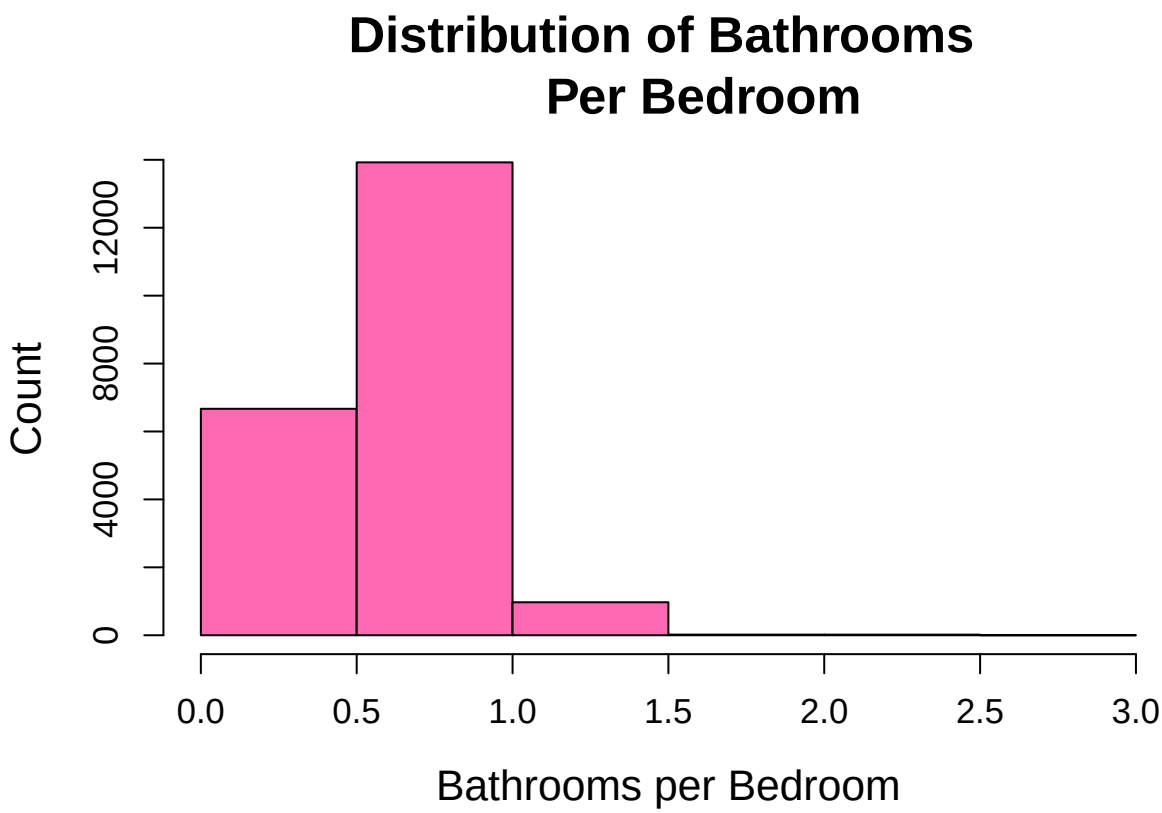
Finally, for years_since_reno, I restricted the distirbution to homes that were actually renovated and removed negative or missing values. Among renovated properties, most updates occured within the last 10-30 years, with relatively few very recent (0-5 years) or very old (40+ years) renovations. This suggests that when renovations do happen, they are often neither brand-new flips nor extremely dated (although, who doesn't love a home with character); instead, many homes sit in a middle band of "relatively recent" updates. Taken together, these distributions support my desirability story: large, recently updated homes with generous bathroom coverage and premium views exist, but they sit in the long right tails of the histograms rather than representing the typical house. In the next section, I use correlations to quantify how these desirability variables relate to sale price overall, and then compare how those relationships differ between wealthy and non-wealthy neighborhoods.

```r
price_hist <- house_sales$price

hist(price_hist, breaks = 40, col = "hotpink", border = "black",
main = "Distribution of Sale Prices", xlab = "House Prices",
ylab = "Number of Houses", xaxt = "n", cex.main = 1.6, cex.lab = 1.3)

ticks <- pretty(price_hist, n = 5)

axis(1, at = ticks, labels = format(ticks, big.mark = ","), cex.axis = 1.1,
     las = 1)
```

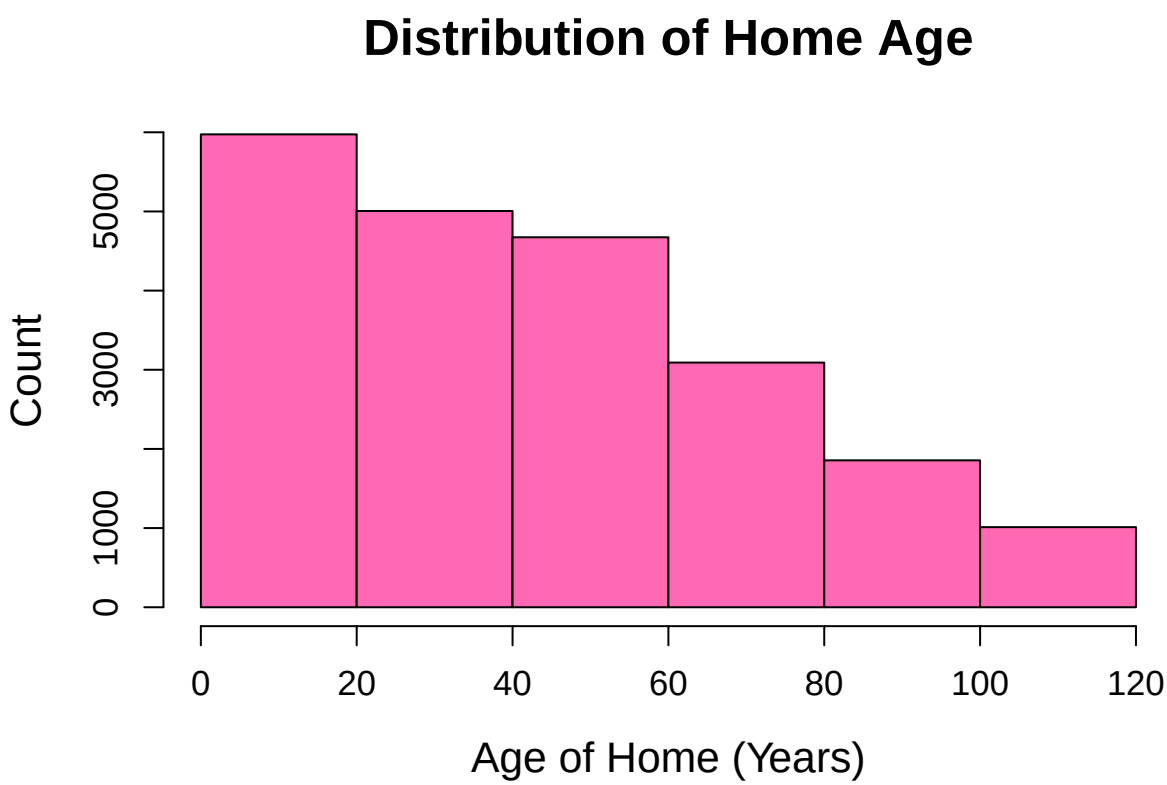



```r
bedroom_bathroom_hist <- house_sales$bed_bath_total
bedroom_bathroom_hist <- bedroom_bathroom_hist[!is.na(bedroom_bathroom_hist)]

hist(bedroom_bathroom_hist, breaks = pretty(bedroom_bathroom_hist),
     col = "hotpink", border = "black",
     main = "Distribution of Total Bedrooms & Bathrooms",
     xlab = "Bedrooms & Bathrooms", ylab = "Count", cex.main = 1.6,
     cex.lab = 1.3, cex.axis = 1.1)
```

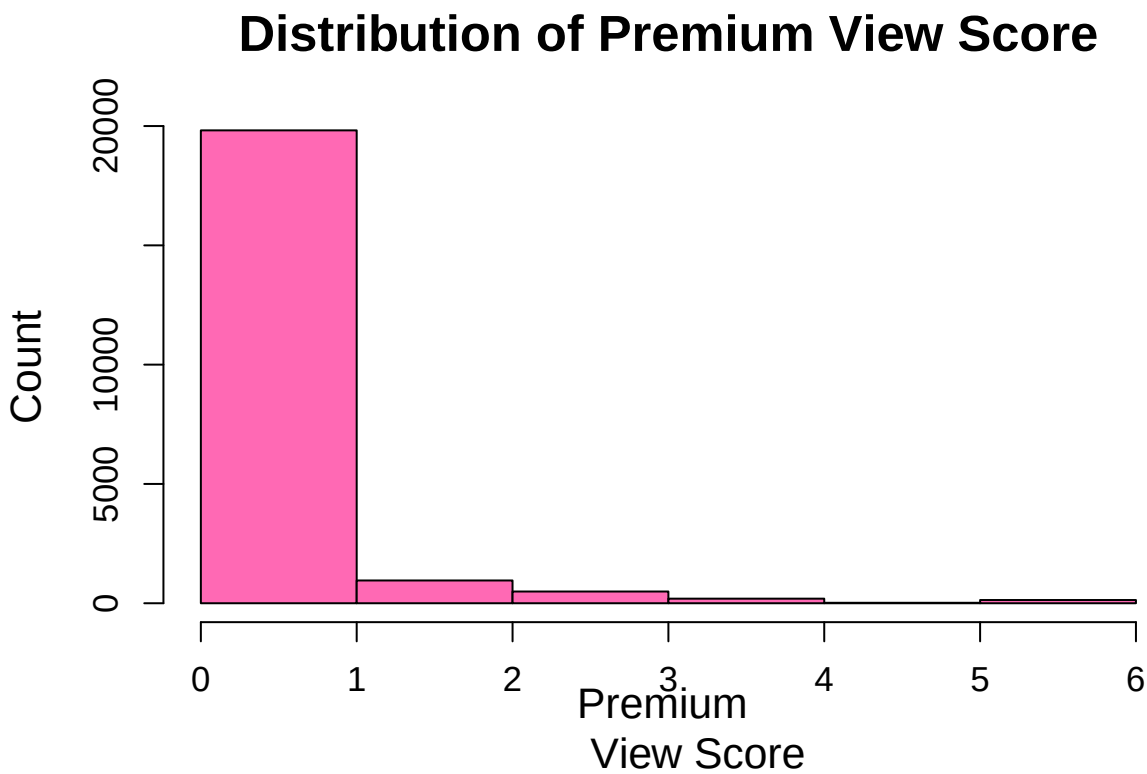
```
bathrooms_per_bedroom_hist <- house_sales$bath_per_bed
bathrooms_per_bedroom_hist <-
  bathrooms_per_bedroom_hist[!is.na(bathrooms_per_bedroom_hist)]

hist(bathrooms_per_bedroom_hist, breaks = pretty(bathrooms_per_bedroom_hist),
     col = "hotpink", border = "black", main = "Distribution of Bathrooms
     Per Bedroom", xlab = "Bathrooms per Bedroom", ylab = "Count", cex.main =
       1.6, cex.lab = 1.3, cex.axis = 1.1)
```



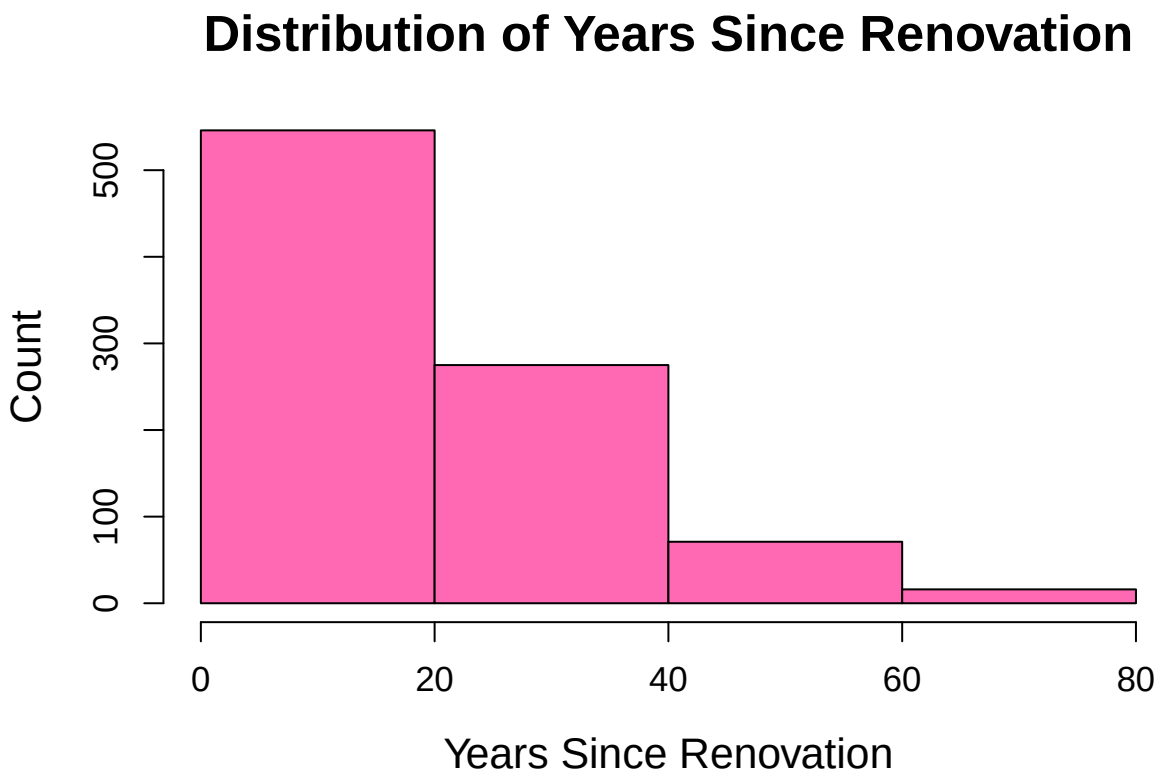

```
house_sales$age[house_sales$age < 0] <- 0

home_age <- house_sales$age
home_age <- home_age[!is.na(home_age)]

hist(home_age, breaks = pretty(home_age), col = "hotpink", border = "black",
     main = "Distribution of Home Age", xlab = "Age of Home (Years)", ylab =
       "Count", cex.main = 1.6, cex.lab = 1.3, cex.axis = 1.1)
```




```
premium_view_score_hist <- house_sales$premium_view_score
premium_view_score_hist <- premium_view_score_hist[!is.na(premium_view_score_hist)]

hist(premium_view_score_hist, breaks = seq(floor(min(premium_view_score_hist)),
                                           ceiling(max(premium_view_score_hist)),
                                           by = 1), col = "hotpink", border =
       "black", main = "Distribution of Premium View Score", xlab = "Premium
     View Score", ylab = "Count", cex.main = 1.6, cex.lab = 1.3, cex.axis = 1.1 )
```




```
years_since_reno_hist <- house_sales$years_since_reno[house_sales$renovated == 1]
years_since_reno_hist <- years_since_reno_hist[ years_since_reno_hist >= 0 &
                                                !is.na( years_since_reno_hist)]

hist(years_since_reno_hist, breaks = pretty(years_since_reno_hist), col = "hotpink",
     border = "black", main = "Distribution of Years Since Renovation", xlab =
  "Years Since Renovation", ylab = "Count", cex.main = 1.6, cex.lab = 1.3,
cex.axis = 1.1)
```



0.11 Correlations & Identifying Independent and Dependent Variables

After mapping out the desirability distributions, I next looked at how these variables move together and which ones should play the role of predictors versus outcomes. At the house level, I treat sale price as the primary outcome: it is the thing prospective homeowners ultimately care about and the quantity I want to explain. My engineered desirability features - overall size (bed_bath_total), bathroom crowdedness (bath_per_bed), view quality (premium_view_score), age, and renovation timing (years_since_reno) - are used as independent variables.

The correlation matrix confirms and sharpens the story from the histograms. Price is moderately correlated with both total bedrooms-plus-bathrooms (r ~0.41), which fits the idea that large homes with better views sit in the "fancy" tail of the price distribution. Bathroom coverage and renovation timing show weaker but still meaningful relationships with price (r ~0.26 for bath_per_bed, r ~-0.13 for years_since_reno), suggesting that eextra bathrooms and more recent updates make a small dent in addition to the existing home size and view. Age by itself has almost no linear relatoinship with price (r ~-0.05), consistent with the idea that both very old and very new homes can be expensive depending on their other amenities.

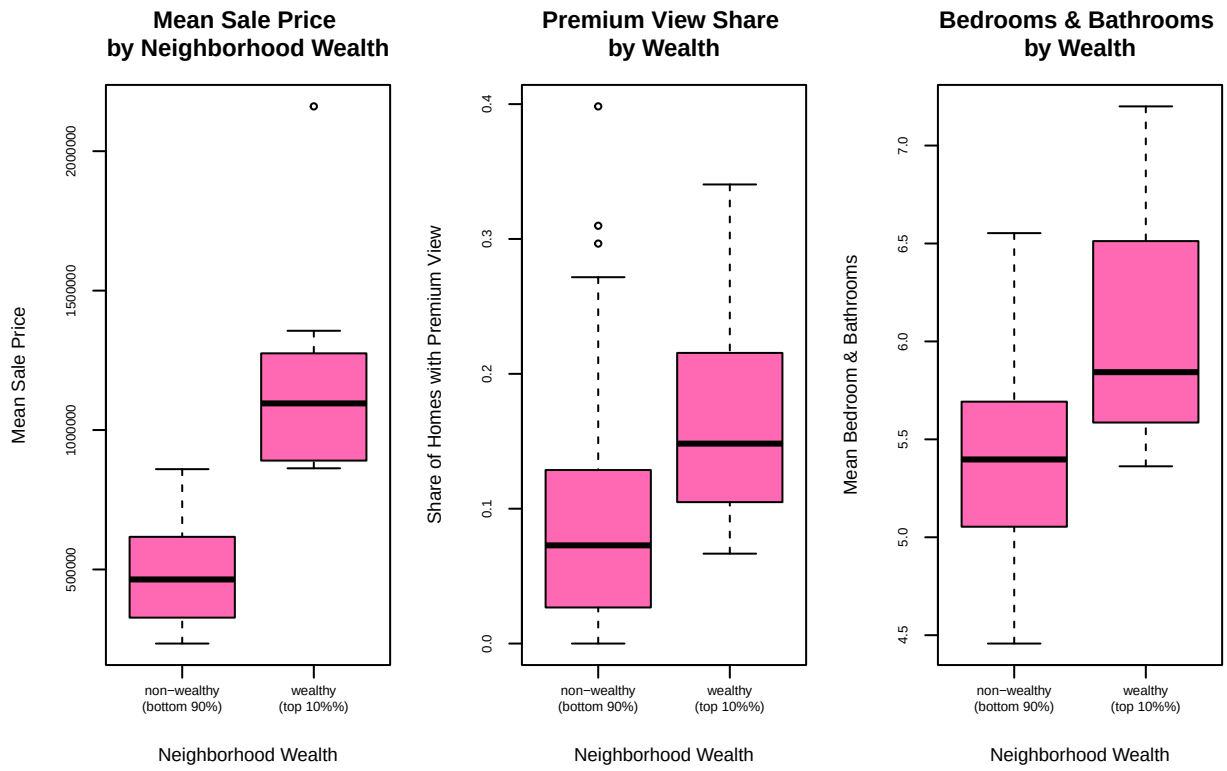
At the zipcode level, I summarize these desirability features and compare them across my discretized wealthy vs non-wealthy groupings. The boxplots show that wealthy zipcodes not only have higher mean prices but also larger homes and a higher share of premium views, so the neighborhood-level pattern mirrors the house-level correlations. Together, the distributions, correlations, boxplots, and scatterplots tell a coherent story: desirability variables behave like plausible "drivers" or price.

What I don't yet know is whether the market naturally organizes itself into distinct clusters of desirability. If I ignore sale price and just look at the feature space, do homes fall into clusters like "starter homes", "middle market homes", and "premium homes"? And do those clusters line up with price and wealthy neighborhoods the way I think they should? That is the question I take up in my final step.

```r
desirability_correlation_variables <- house_sales %>% select(price, bed_bath_total,
                                                             bath_per_bed, age,
                                                             premium_view_score,
                                                             years_since_reno)

desirability_correlations <- cor(desirability_correlation_variables,
                                 use = "pairwise.complete.obs")

desirability_correlations
```

```
##                          price bed_bath_total bath_per_bed         age
## price               1.00000000      0.4616112    0.2639036 -0.05395613
## bed_bath_total      0.46161121      1.0000000    0.1737652 -0.35488870
## bath_per_bed        0.26390360      0.1737652    1.0000000 -0.39613810
## age                -0.05395613     -0.3548887   -0.3961381  1.00000000
## premium_view_score  0.41204691      0.1376045    0.1250554  0.05345984
## years_since_reno   -0.12815325     -0.2274874   -0.1202694  0.25019483
##                    premium_view_score years_since_reno
## price                      0.41204691      -0.12815325
## bed_bath_total             0.13760448      -0.22748740
## bath_per_bed               0.12505535      -0.12026937
## age                        0.05345984       0.25019483
## premium_view_score         1.00000000       0.07345308
## years_since_reno           0.07345308       1.00000000
```

```r
zipcode_summary$wealthy_neighborhoods <- factor(zipcode_summary$wealthy_neighborhoods,
                                                levels = c("non-wealthy (bottom 90%)",
                                                           "wealthy (top 10%)"),
                                                labels = c("non-wealthy\n(bottom 90%)",
                                                           "wealthy\n(top 10%%)"))

par(mfrow = c(1,3), mar = c(7, 4, 4, 1) + 0.1, cex.axis = 0.6, cex.lab = 0.9,
    cex.main = 1.1)

boxplot(mean_price ~ wealthy_neighborhoods, data = zipcode_summary,
        col = "hotpink",
        border = "black", main = "Mean Sale Price \nby Neighborhood Wealth",
        xlab = "Neighborhood Wealth", ylab = "Mean Sale Price")

boxplot(premium_view_share ~ wealthy_neighborhoods, data = zipcode_summary,
        col = "hotpink",border = "black",
        main = "Premium View Share \nby Wealth", xlab = "Neighborhood Wealth",
        ylab = "Share of Homes with Premium View")

boxplot(mean_bed_bath_total ~ wealthy_neighborhoods, data = zipcode_summary,
        col = "hotpink", border = "black",
        main = "Bedrooms & Bathrooms \nby Wealth", xlab = "Neighborhood Wealth",
        ylab = "Mean Bedroom & Bathrooms")
```

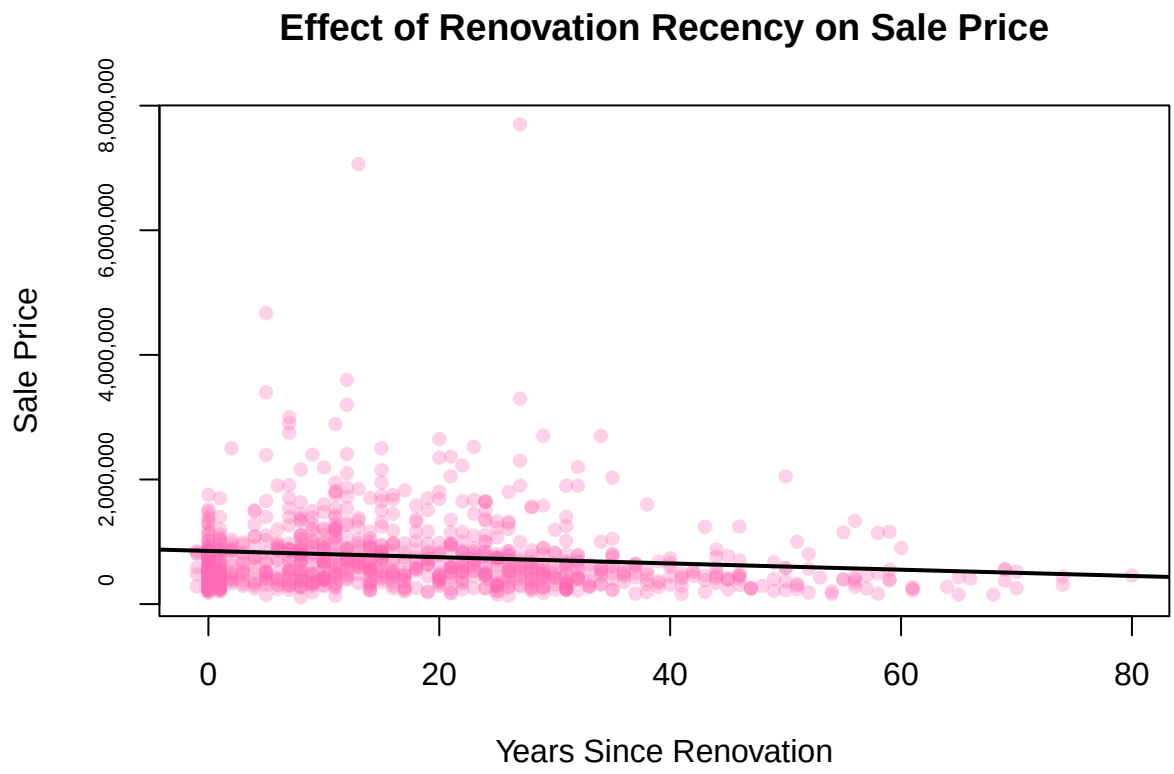



```r
par(mfrow = c(1,1 ))
```

```r
plot(house_sales$bed_bath_total, house_sales$price, pch = 16,
     col = rgb(1, 0.41, 0.71, 0.3), xlab = "Total Bedrooms & Bathrooms",
     ylab = "Sale Price", main = "Relationship Between Home Size and Sale Price",
     yaxt = "n")

price_ticks <- pretty(house_sales$price)
axis(2, at = price_ticks, labels = format(price_ticks, big.mark = ","),
     cex.axis = 0.7)

abline(lm(price ~ bed_bath_total, data = house_sales), lwd = 2)
```

```r
plot(house_sales$premium_view_score, house_sales$price, pch = 16,
     col = rgb(1, 0.41, 0.71, 0.3), xlab = "Premium View Score",
     ylab = "Sale Price",
     main = "Relationship Between View Quality and Sale Price", yaxt = "n")

price_ticks <- pretty(house_sales$price)
axis(2, at = price_ticks, labels = format(price_ticks, big.mark = ","),
     cex.axis = 0.7)

abline(lm(price ~ premium_view_score, data = house_sales), lwd = 2)
```

```r
reno_data <- subset(house_sales, !is.na(years_since_reno))

plot(reno_data$years_since_reno, reno_data$price, pch = 16,
     col = rgb(1, 0.41, 0.71, 0.3), xlab = "Years Since Renovation",
     ylab = "Sale Price", main = "Effect of Renovation Recency on Sale Price",
     yaxt = "n")

price_ticks <- pretty(house_sales$price)
axis(2, at = price_ticks, labels = format(price_ticks, big.mark = ","),
     cex.axis = 0.7)

abline(lm(price ~ years_since_reno, data = reno_data), lwd = 2)
```



0.12 Unsupervised Learning: Does Desirability Garner A Premium?

To close the analysis, I switched from a supervised mindset to an unsupervised one. Instead of asking "how well can we predict price from desirability?"; I asked a deeper question: "if I completely ignore sale price and neighborhood wealth and only look at the desirability features, does the market naturally sort homes into distinct clusters? And if so, do those clusters line up with the price and wealthy zipcode story my data has been building thus far?"

I built a clustering dataset at the house level using four standardized features: total bedrooms and bathrooms, bathrooms per bedroom, premium view score, and years since renovation. Running k-means with three clusters and multiple random starts produced a solution that explains about 41% of the variance in the k-means analysis. When I bring price back into the conversation for interpretation purposes, the clusters fall into clear categories; one group of smaller, older, low-view homes with the loweest prices; a mid-high group of larger, recently renovated but mostly non-view homes; and a high-end group of big, bathroom-rich houses with high premium view scores and the highest prices. The clustering algorithm never saw price, yet it recreated the "starter / middle market / premium" structure implied by my histograms and correlations.

I then linked these house-level clusters back to my wealthy vs non-wealthy zipcode labels. As only 10% of zipcodes are defined as wealthy, 10% is the natural baseline share for each cluster if desirability clusters were randomly scattered across neighborhoods. Instead, the low-desirability cluster has only about 7% of its homes in wealthy zipcodes, while the two higher-desirability clusters come in around 14% and 19% wealthy, respectively. In other words, premium and renovated clusters are over-represented in wealthy neighborhoods, and the modest cluster is under-represented. The "missing" low-desirability homes in rich areas are part of the evidence: it's a 1-p argument that strengthens the story rather than weakening it.

For evaluation, I treat the 41% variance as an internal quality check that the clusters are non-trivial, and the alignment with price and neighborhood wealth as an external validity check. Without ever being told what

"expensive" or "wealthy" means, k-means rediscovers clusters that behave exactly like clusters of housing desirability: as you move from the modest cluster to the premium view-heavy and renovated clusters, both prices and the share of wealthy neighborhoods rise. The unsupervised learning step doesn't overturn my earlier conclusions; it stress-tests them and reaffirms that desirability does, in fact, garner a premium in this housing market.

```
cluster_desirability_data <- house_sales %>%
  select(price, bed_bath_total, bath_per_bed, premium_view_score,
         years_since_reno, zipcode) %>% na.omit()

desirability_matrix <- cluster_desirability_data %>% select(bed_bath_total,
                                                            bath_per_bed,
                                                            premium_view_score,
                                                            years_since_reno)

desirability_scaled <- scale(desirability_matrix)

set.seed(123)

k_means_3 <- kmeans(desirability_scaled, centers = 3, nstart = 25)



cluster_desirability_data$cluster_3 <- factor(k_means_3$cluster, levels = 1:3,
                                              labels = c("Cluster 1", "Cluster 2",
                                                         "Cluster 3"))

total_within <- k_means_3$tot.withinss
proportion_variance <- k_means_3$betweenss / k_means_3$totss

total_within
```

```
## [1] 2150.452
```

```
proportion_variance
```

```
## [1] 0.4111577
```

```
cluster_desirability_data_summary <- cluster_desirability_data %>%
  group_by(cluster_3) %>% summarise(n = dplyr::n(), mean_price = mean(price),
                                    mean_bed_bath_total = mean(bed_bath_total),
                                    mean_bath_per_bed = mean(bath_per_bed),
                                    mean_view_score = mean(premium_view_score),
                                    mean_years_since_reno = mean(years_since_reno),
                                    .groups = "drop")

cluster_desirability_data_summary
```

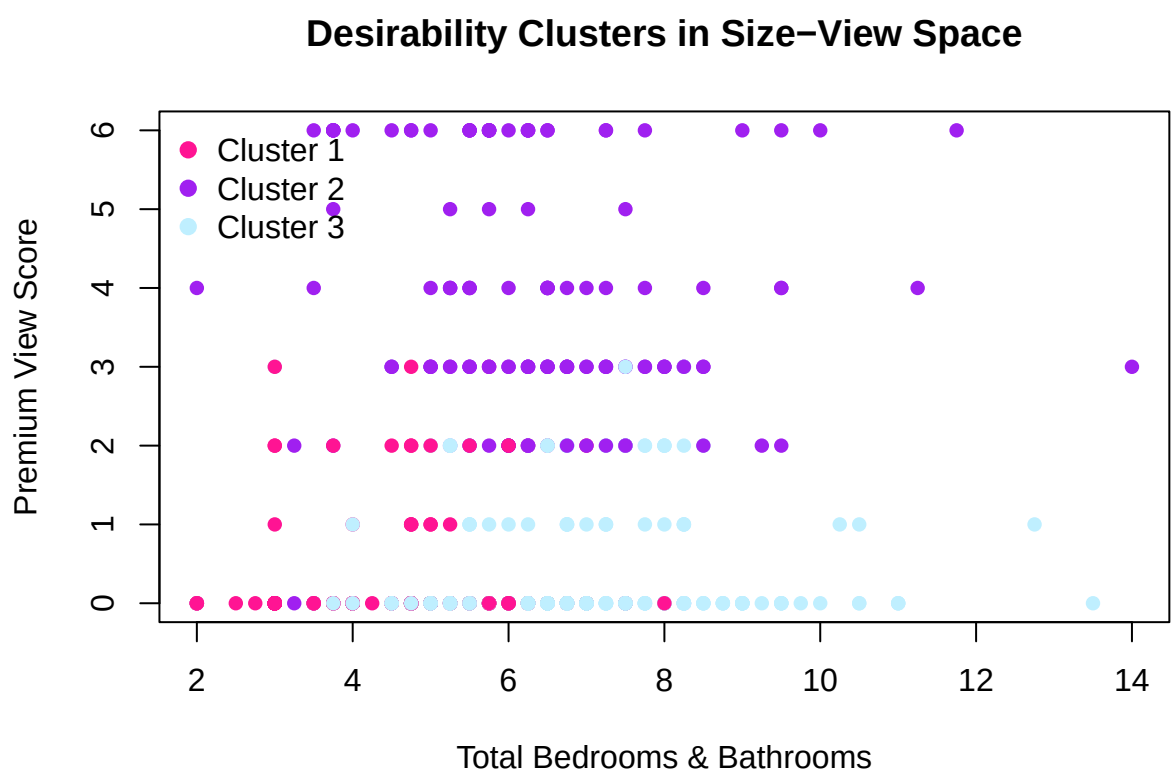
```
## # A tibble: 3 x 7
##   cluster_3     n mean_price mean_bed_bath_total mean_bath_per_bed
##   <fct>     <int>      <dbl>               <dbl>             <dbl>
## 1 Cluster 1   259    451228.                4.56             0.562
```

```
## 2 Cluster 2   130   1349453.                6.35             0.895
## 3 Cluster 3   525    767028.                6.15             0.692
## # i 2 more variables: mean_view_score <dbl>, mean_years_since_reno <dbl>
```

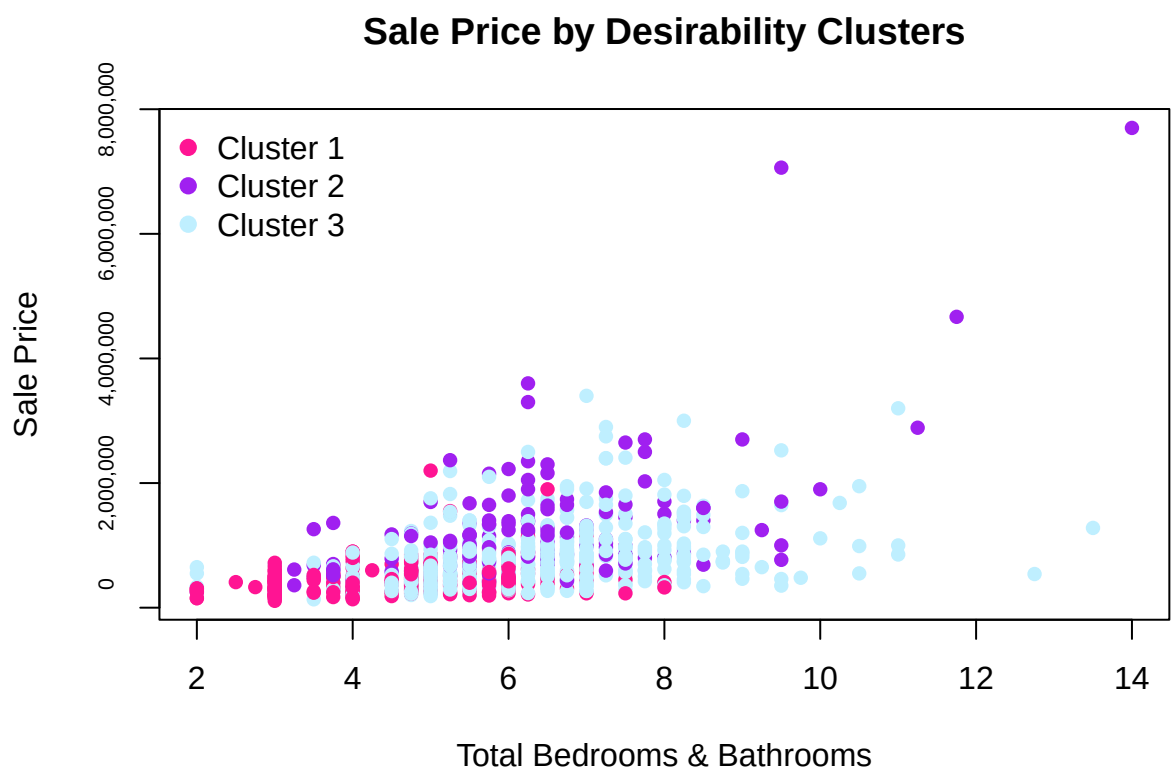
```
cluster_wealth_crosstab <- table(cluster_desirability_data$cluster_3, zipcode_summary$wealthy_neighborhc


cluster_wealth_crosstab
```

```
##
##              non-wealthy\n(bottom 90%) wealthy\n(top 10%%)
##   Cluster 1                        240                  19
##   Cluster 2                        112                  18
##   Cluster 3                        426                  99
```

```
cluster_colors <- c("deeppink", "purple", "lightblue1")

plot(cluster_desirability_data$bed_bath_total,
     cluster_desirability_data$premium_view_score,
     col = cluster_colors[cluster_desirability_data$cluster_3],
     pch = 16, xlab = "Total Bedrooms & Bathrooms", ylab = "Premium View Score",
     main = "Desirability Clusters in Size-View Space")

legend("topleft", legend = c("Cluster 1", "Cluster 2", "Cluster 3"),
       col = cluster_colors, pch = 16, pt.cex = 1.2, bty = "n")
```



```r
plot(cluster_desirability_data$bed_bath_total, cluster_desirability_data$price,
     col = cluster_colors[cluster_desirability_data$cluster_3], pch = 16,
     xlab = "Total Bedrooms & Bathrooms", ylab = "Sale Price",
     main = "Sale Price by Desirability Clusters", yaxt = "n")

price_ticks <- pretty(house_sales$price)
axis(2, at = price_ticks, labels = format(price_ticks, big.mark = ","),
     cex.axis = 0.7)

legend("topleft", legend = c("Cluster 1", "Cluster 2", "Cluster 3"),
       col = cluster_colors, pch = 16, pt.cex = 1.2, bty = "n")
```



0.13 References

Bobbitt, Z. (2020, November 18). *Understanding Heteroscedasticity in Regression Analysis.* Statology. https://www.statology.org/heteroscedasticity-regression/

Bhalla, D. (n.d.). *IF ELSE CONDITIONS IN R (WITH EXAMPLES).* https://www.listendata.com/2017/03/if-else-in-r.html

Data Carpentry. (n.d.). *Aggregating and analyzing data with dplyr.* https://datacarpentry.github.io/R-genomics/04-dplyr.html

Galli, S. (2022, July 4). *Data discretization in machine learning* Train in Data. https://www.blog.trainindata.com/data-discretization-in-machine-learning/

Geeksforgeeks. (2025, November 29). *Aggregation in Data Mining.* https://www.geeksforgeeks.org/data-analysis/aggregation-in-data-mining/

Geeksforgeeks.(2025, November 08). *What is Feature Engineering?* https://www.geeksforgeeks.org/machine-learning/what-is-feature-engineering/

Huynh, W. (2019, August 06). *Advanced Data Management: ifelse().*
R for Graduate Students. https://bookdown.org/yih_huynh/Guide-to-R-Book/ifelse.html

Maddula, S. (2025, March 19). *Feature Engineering in Machine Learning: A Practical Guide.* https://www.datacamp.com/tutorial/feature-engineering

Mounish, V. (2024, November 22). *What is Discretization in Machine Learning?* Analytics Vidhya. https://www.analyticsvidhya.com/blog/2024/11/discretization/

OpenAI. (2025). *ChatGPT[Large language model].* https://chat.openai.com

R bloggers. (2022, March 16). *R Data types 101, or What kind of data do I have?* https://www.r-bloggers.com/2022/03/r-data-types-101-or-what-kind-of-data-do-i-have/

Sanderson, S.P. (2024, December 2). *How to Replace Missing Values in R: A Comprehensive Guide.* https://www.spsanderson.com/steveondata/posts/2024-12-02/

Smola, A. and Vishwanathan, S. (2008). *Introduction to Machine Learning.* Cambridge University Press. https://alex.smola.org/drafts/thebook.pdf

Wikipedia. (n.d.). *Feature engineering* https://en.wikipedia.org/wiki/Feature_engineering#Clustering

Wentworth Studio. (n.d.). *Mid-Century Modern Style (1935–1975).* https://www.wentworthstudio.com/historic-styles/mid-century-modern

0.14 Artifical Intelligence Disclosure

I used GPT as a support tool for this project similar to an advanced search engine. GPT helped me recall R syntax, debug plotting choices (e.g., labels, margins, colors), and structure the k-means clustering code and evaluation steps. All analytical decisions, interpretations, and final edits are my own.